

Data Storage and Persistence

From Hardware to File I/O and Data Privacy

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

To understand the fundamental differences between volatile memory (RAM) and persistent storage. We will explore the underlying hardware (HDDs and SSDs), the abstractions provided by the Operating System (File Systems), master the techniques for reading and writing both text and binary data in Python, and review the critical legal obligations of data handling (GDPR).

Lecture Overview

- 1 The Problem with Memory (RAM)
- 2 Persistent Storage Hardware
- 3 The Operating System Abstraction: File Systems
- 4 Working with Text Files in Python
- 5 Storing Binary Data
- 6 Storage at Scale (Big Data)
- 7 Data Privacy and Legal Obligations

The Problem with Memory (RAM)

The Problem with Memory (RAM)

Where Does a Program Store Data?

When we execute a program (e.g., in Python), we store data in variables (lists, dictionaries, integers). Where are these physically located?

The Location: Random Access Memory (RAM)

- Variables reside in the computer's RAM (integrated circuit chips).
- RAM is designed for high-speed access by the CPU during execution.

The Interface

While RAM stores billions of individual bits (1s and 0s), the smallest unit the CPU typically addresses is a **byte** (8 bits). This allows us to conceptualise memory as storing numbers and data structures.

Where Does a Program Store Data?

When we execute a program (e.g., in Python), we store data in variables (lists, dictionaries, integers). Where are these physically located?

The Location: Random Access Memory (RAM)

- Variables reside in the computer's RAM (integrated circuit chips).
- RAM is designed for high-speed access by the CPU during execution.

The Interface

While RAM stores billions of individual bits (1s and 0s), the smallest unit the CPU typically addresses is a **byte** (8 bits). This allows us to conceptualise memory as storing numbers and data structures.

Where Does a Program Store Data?

The Location: Random Access Memory (RAM)

- Variables reside in the computer's RAM (integrated circuit chips).
- RAM is designed for high-speed access by the CPU during execution.

The Interface

While RAM stores billions of individual bits (1s and 0s), the smallest unit the CPU typically addresses is a **byte** (8 bits). This allows us to conceptualise memory as storing numbers and data structures.

The Fundamental Question

What happens to the data stored in RAM when the program finishes or the computer is turned off?

The Nature of DRAM: A Forgetful Medium

The specific type of RAM used in most computers is Dynamic RAM (DRAM).

The Mechanism: Capacitors and Charge

DRAM stores each bit of data in a separate tiny capacitor within an integrated circuit. The electrical charge in the capacitor determines the bit's value (1 or 0).

The Problem: Charge Leakage

These capacitors are imperfect and leak charge rapidly. DRAM typically cannot retain data for more than about **100 milliseconds** before the charge dissipates and the data is lost.

The Nature of DRAM: A Forgetful Medium

The specific type of RAM used in most computers is Dynamic RAM (DRAM).

The Mechanism: Capacitors and Charge

DRAM stores each bit of data in a separate tiny capacitor within an integrated circuit. The electrical charge in the capacitor determines the bit's value (1 or 0).

The Problem: Charge Leakage

These capacitors are imperfect and leak charge rapidly. DRAM typically cannot retain data for more than about **100 milliseconds** before the charge dissipates and the data is lost.

The Volatility of RAM

The Solution: The Refresh Cycle

To counteract charge leakage, the hardware memory controller continuously performs a **refresh cycle** in the background. It reads the memory contents and writes them back, recharging the capacitors.

The Consequence: Volatility

This refresh process requires constant electrical power.

- As soon as the power is turned off, the refresh stops.
- The charge drains away, and all contents of the RAM are permanently lost.

Definition

RAM is **Volatile Memory**: it requires power to maintain the stored information.

The Volatility of RAM

The Solution: The Refresh Cycle

To counteract charge leakage, the hardware memory controller continuously performs a **refresh cycle** in the background. It reads the memory contents and writes them back, recharging the capacitors.

The Consequence: Volatility

This refresh process requires constant electrical power.

- As soon as the power is turned off, the refresh stops.
- The charge drains away, and all contents of the RAM are permanently lost.

Definition

RAM is **Volatile Memory**: it requires power to maintain the stored information.

The Volatility of RAM

The Solution: The Refresh Cycle

To counteract charge leakage, the hardware memory controller continuously performs a **refresh cycle** in the background. It reads the memory contents and writes them back, recharging the capacitors.

The Consequence: Volatility

This refresh process requires constant electrical power.

- As soon as the power is turned off, the refresh stops.
- The charge drains away, and all contents of the RAM are permanently lost.

Definition

RAM is **Volatile Memory**: it requires power to maintain the stored information.

Operating System Memory Management

Even if the computer remains powered on, data stored in RAM by a program is transient.

The OS as the Memory Owner

- The Operating System (OS) owns and manages the physical RAM.
- The OS "loans" memory segments to a program only while it is running.

Memory Reclamation

As soon as a program terminates, the OS reclaims the loaned memory. That memory is then reallocated to other programs. The previous contents are overwritten and lost.

The Need for Persistence

We require a mechanism for **Persistent Storage**—storing data beyond the lifetime of the program and the computer's power cycle.

Operating System Memory Management

Even if the computer remains powered on, data stored in RAM by a program is transient.

The OS as the Memory Owner

- The Operating System (OS) owns and manages the physical RAM.
- The OS "loans" memory segments to a program only while it is running.

Memory Reclamation

As soon as a program terminates, the OS reclaims the loaned memory. That memory is then reallocated to other programs. The previous contents are overwritten and lost.

The Need for Persistence

We require a mechanism for **Persistent Storage**—storing data beyond the lifetime of the program and the computer's power cycle.

Operating System Memory Management

Even if the computer remains powered on, data stored in RAM by a program is transient.

The OS as the Memory Owner

- The Operating System (OS) owns and manages the physical RAM.
- The OS "loans" memory segments to a program only while it is running.

Memory Reclamation

As soon as a program terminates, the OS reclaims the loaned memory. That memory is then reallocated to other programs. The previous contents are overwritten and lost.

The Need for Persistence

We require a mechanism for **Persistent Storage**—storing data beyond the lifetime of the program and the computer's power cycle.

Persistent Storage Hardware

Persistent Storage Hardware

The Landscape of Persistent Storage

We rely on non-volatile storage devices to retain data when the power is off. There are two primary technologies in modern computing.

1. Hard Disk Drives (HDDs)

- Mechanical, magnetic storage.
- Dominant technology for decades.
- High storage density (very large capacity) and lower cost per GB.

2. Solid-State Drives (SSDs)

- Electronic storage (Flash memory).
- No moving parts.
- Significantly faster access times.

The Landscape of Persistent Storage

We rely on non-volatile storage devices to retain data when the power is off. There are two primary technologies in modern computing.

1. Hard Disk Drives (HDDs)

- Mechanical, magnetic storage.
- Dominant technology for decades.
- High storage density (very large capacity) and lower cost per GB.

2. Solid-State Drives (SSDs)

- Electronic storage (Flash memory).
- No moving parts.
- Significantly faster access times.

Hard Disk Drives (HDDs): Mechanics

HDDs rely on mechanical movement and magnetism.

Components

Platters: Rigid disks coated in a thin magnetic layer (e.g., iron oxide). Data is stored by changing the magnetic state of this layer.

Spindle: Rotates the platters at high speeds (e.g., 7200 RPM).

Read/Write Head: Mounted on a mechanical arm, this head moves across the surface of the spinning platter to read or modify the magnetic state.

Performance Implication: Latency

Accessing data requires physical movement: moving the arm (seek time) and waiting for the disk to rotate (rotational latency). This makes random access slow compared to electronic memory.

Hard Disk Drives (HDDs): Mechanics

Components

Platters: Rigid disks coated in a thin magnetic layer (e.g., iron oxide). Data is stored by changing the magnetic state of this layer.

Spindle: Rotates the platters at high speeds (e.g., 7200 RPM).

Read/Write Head: Mounted on a mechanical arm, this head moves across the surface of the spinning platter to read or modify the magnetic state.

Performance Implication: Latency

Accessing data requires physical movement: moving the arm (seek time) and waiting for the disk to rotate (rotational latency). This makes random access slow compared to electronic memory.

HDDs: Blocks and Error Correction

The physical surface of a disk platter is not perfect. Imperfections can cause bits to be stored incorrectly.

The Block Abstraction

Drives do not store data byte-by-byte. Instead, they operate on **blocks** (or sectors).

- A typical block size is 512 bytes or 4KB.
- The block is the smallest unit of data that can be read or written.

Error Correcting Codes (ECC)

At the end of each block, the drive hardware stores an Error Correcting Code (ECC). When the block is read back, the drive controller uses the ECC to detect and automatically correct minor data corruption before returning the data to the OS.

HDDs: Blocks and Error Correction

The physical surface of a disk platter is not perfect. Imperfections can cause bits to be stored incorrectly.

The Block Abstraction

Drives do not store data byte-by-byte. Instead, they operate on **blocks** (or sectors).

- A typical block size is 512 bytes or 4KB.
- The block is the smallest unit of data that can be read or written.

Error Correcting Codes (ECC)

At the end of each block, the drive hardware stores an Error Correcting Code (ECC). When the block is read back, the drive controller uses the ECC to detect and automatically correct minor data corruption before returning the data to the OS.

Solid-State Drives (SSDs)

SSDs utilize flash memory technology and have no moving parts.

The Mechanism: Flash Storage

- Uses **floating gate transistors**.
- Data is stored by trapping electrical charge directly within these transistors.

Performance Advantage

- **Random Access:** Much faster than HDDs as data can be retrieved electronically without mechanical movement.
- **Note:** SSDs are still significantly slower than DRAM.

Solid-State Drives (SSDs)

SSDs utilize flash memory technology and have no moving parts.

The Mechanism: Flash Storage

- Uses **floating gate transistors**.
- Data is stored by trapping electrical charge directly within these transistors.

Performance Advantage

- **Random Access:** Much faster than HDDs as data can be retrieved electronically without mechanical movement.
- **Note:** SSDs are still significantly slower than DRAM.

Solid-State Drives (SSDs)

The Mechanism: Flash Storage

- Uses **floating gate transistors**.
- Data is stored by trapping electrical charge directly within these transistors.

Performance Advantage

- **Random Access:** Much faster than HDDs as data can be retrieved electronically without mechanical movement.
- **Note:** SSDs are still significantly slower than DRAM.

The Drawback: Write Endurance

Flash storage has limited lifespan. Writing to same memory cell repeatedly causes it to wear out and eventually fail.

SSDs: Wear Levelling

To mitigate the limited write endurance of flash memory, SSDs employ sophisticated management techniques.

The SSD Controller and Wear Levelling

The SSD controller hides the physical complexity and performs **wear levelling**.

- It constantly rotates where data is physically stored on the flash cells.
- This ensures that writes are distributed evenly across the drive, preventing any single cell from wearing out prematurely.

The Interface Remains Consistent

Despite the internal complexity of wear levelling, the SSD still presents a standard block-based interface to the Operating System, just like an HDD.

SSDs: Wear Levelling

To mitigate the limited write endurance of flash memory, SSDs employ sophisticated management techniques.

The SSD Controller and Wear Levelling

The SSD controller hides the physical complexity and performs **wear levelling**.

- It constantly rotates where data is physically stored on the flash cells.
- This ensures that writes are distributed evenly across the drive, preventing any single cell from wearing out prematurely.

The Interface Remains Consistent

Despite the internal complexity of wear levelling, the SSD still presents a standard block-based interface to the Operating System, just like an HDD.

The Operating System Abstraction: File Systems

The Operating System Abstraction: File Systems

The Need for Abstraction

Directly managing hardware blocks (e.g., "read block 12345 from drive 1") is impractical for programmers.

Challenges of Direct Hardware Access

- Requires precise knowledge of the hardware geometry and block locations.
- Hardware-specific commands (different for HDDs vs SSDs, different manufacturers).
- No mechanism for organizing or naming data meaningfully.

The Role of the Operating System

The OS provides a crucial abstraction layer: the **File System** (e.g., NTFS, ext4, APFS), to hide the hardware complexity and provide a usable interface.

The Need for Abstraction

Directly managing hardware blocks (e.g., "read block 12345 from drive 1") is impractical for programmers.

Challenges of Direct Hardware Access

- Requires precise knowledge of the hardware geometry and block locations.
- Hardware-specific commands (different for HDDs vs SSDs, different manufacturers).
- No mechanism for organizing or naming data meaningfully.

The Role of the Operating System

The OS provides a crucial abstraction layer: the **File System** (e.g., NTFS, ext4, APFS), to hide the hardware complexity and provide a usable interface.

The Functions of a File System

The File System serves several primary purposes:

1. Hierarchical Namespace

It provides a structure of **directories** (folders) and **files**. This allows users and programs to organize data logically and assign meaningful names (e.g., '/users/admin/data.txt').

2. Translation and Mapping

The OS is responsible for translating the logical file operations into physical block operations on the storage device. It maintains metadata to track which blocks belong to which file.

The Functions of a File System

The File System serves several primary purposes:

1. Hierarchical Namespace

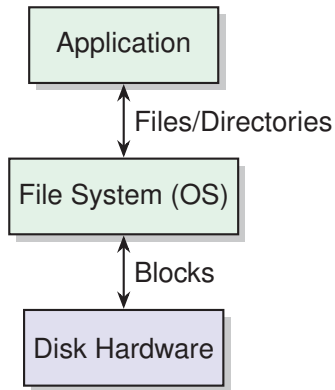
It provides a structure of **directories** (folders) and **files**. This allows users and programs to organize data logically and assign meaningful names (e.g., '/users/admin/data.txt').

2. Translation and Mapping

The OS is responsible for translating the logical file operations into physical block operations on the storage device. It maintains metadata to track which blocks belong to which file.

The Functions of a File System

The File System serves several primary purposes:



The Definition of a File

What exactly is a file from the perspective of the OS and the programmer?

A Sequence of Bytes

In most modern operating systems, a file is simply treated as a linear sequence of bytes stored in persistent storage.

No Inherent Semantics

The OS does not impose any meaning (semantics) or structure on these bytes. It does not know if the bytes represent text, an image, or numerical data.

The Program's Responsibility

It is up to the application program to interpret the sequence of bytes and give them meaning (e.g., by using specific encodings or file formats).

The Definition of a File

What exactly is a file from the perspective of the OS and the programmer?

A Sequence of Bytes

In most modern operating systems, a file is simply treated as a linear sequence of bytes stored in persistent storage.

No Inherent Semantics

The OS does not impose any meaning (semantics) or structure on these bytes. It does not know if the bytes represent text, an image, or numerical data.

The Program's Responsibility

It is up to the application program to interpret the sequence of bytes and give them meaning (e.g., by using specific encodings or file formats).

The Definition of a File

What exactly is a file from the perspective of the OS and the programmer?

A Sequence of Bytes

In most modern operating systems, a file is simply treated as a linear sequence of bytes stored in persistent storage.

No Inherent Semantics

The OS does not impose any meaning (semantics) or structure on these bytes. It does not know if the bytes represent text, an image, or numerical data.

The Program's Responsibility

It is up to the application program to interpret the sequence of bytes and give them meaning (e.g., by using specific encodings or file formats).

The File Interface Operations

The OS provides a standard set of operations (System Calls) for programs to interact with the file system.

Core Operations

Open/Create: Request access to a file, specifying the mode (read, write).

Read: Retrieve bytes starting from the current position.

Write: Store bytes starting at the current position or appending to the end.

Seek: Move the current read/write position (the "cursor") to a specific byte offset (Random Access).

Close: Release the file, finalizing writes and freeing OS resources.

Working with Text Files in Python

Working with Text Files in Python

Opening a File in Python

Python provides the built-in 'open()' function to interface with the OS file system.

The 'open()' Function

It takes the filename (path) and a mode string as arguments, returning a file object.

```
1 # Syntax: open(filename, mode)
2 file = open("names.txt", "rt")
```

Understanding the Mode String

The mode specifies the intended operation and data interpretation.

'r' (Read): Open for reading only. Fails if the file does not exist.

't' (Text): Interpret the file's bytes as text.

Opening a File in Python

Python provides the built-in 'open()' function to interface with the OS file system.

The 'open()' Function

It takes the filename (path) and a mode string as arguments, returning a file object.

```
1 # Syntax: open(filename, mode)
2 file = open("names.txt", "rt")
```

Understanding the Mode String

The mode specifies the intended operation and data interpretation.

'r' (Read): Open for reading only. Fails if the file does not exist.

't' (Text): Interpret the file's bytes as text.

Opening a File in Python

Python provides the built-in `'open()'` function to interface with the OS file system.

Understanding the Mode String

The mode specifies the intended operation and data interpretation.

`'r'` (Read): Open for reading only. Fails if the file does not exist.

`'t'` (Text): Interpret the file's bytes as text.

Text Encoding

In text mode (`'t'`), Python automatically decodes the bytes from the file into strings. The default encoding (usually UTF-8) is generally appropriate.

Writing Modes

To write data, we must use different access modes.

Write Mode ('w')

```
'open("store.txt", "wt")'
```

- If the file does not exist, it is created.
- **Warning:** If the file already exists, its contents are immediately **truncated** (deleted) before any writing begins.

Append Mode ('a')

```
'open("log.txt", "at")'
```

- If the file does not exist, it is created.
- If the file exists, new data is added to the end of the file without deleting existing content.

Writing Modes

To write data, we must use different access modes.

Write Mode ('w')

```
'open("store.txt", "wt")'
```

- If the file does not exist, it is created.
- **Warning:** If the file already exists, its contents are immediately **truncated** (deleted) before any writing begins.

Append Mode ('a')

```
'open("log.txt", "at")'
```

- If the file does not exist, it is created.
- If the file exists, new data is added to the end of the file without deleting existing content.

Writing Data: The '.write()' Method

The '.write()' method sends a string to the file.

```
1 file = open("store.txt", "wt")
2 file.write("Hello")
3 file.write("World")
4 file.close()
5 # File content: "HelloWorld"
```

Handling Newlines

Unlike `print()`, the `.write()` method does not automatically add newline characters (`\n`). You must add them manually to separate lines.

```
1 # To put them on separate lines:
2 file.write("Hello\n")
3 file.write("World\n")
```

The Critical Importance of Closing Files

You must always call `file.close()` when finished with a file object.

Reason 1: Releasing OS Resources

The OS has a limit on the number of files a program can have open simultaneously. Forgetting to close files can lead to resource exhaustion and crashes.

The Critical Importance of Closing Files

You must always call `'file.close()'` when finished with a file object.

Reason 1: Releasing OS Resources

The OS has a limit on the number of files a program can have open simultaneously. Forgetting to close files can lead to resource exhaustion and crashes.

Reason 2: Buffering and Data Integrity

When writing data, the OS (and Python) often utilizes **buffering**.

- Data written via `'write()'` is initially held in a memory buffer, not immediately written to the physical disk (as disk writes are slow).
- `'file.close()'` forces the buffer to be "flushed," ensuring the data is actually written to the persistent storage.

The Critical Importance of Closing Files

You must always call `file.close()` when finished with a file object.

Consequence

If a program terminates before `close()` is called, data in the buffer may be lost.

Best Practice: The 'with' Statement

Because forgetting to call 'close()' is a common error, Python provides the 'with' statement (Context Manager) for safer file handling.

The 'with' Statement

Guarantees that the file is automatically closed when the indented block is exited, even if errors occur within the block.

```
1 # Recommended approach
2 with open("store.txt", "wt") as file:
3     # The file is open within this indented block
4     file.write("Hello\n")
5     file.write("World\n")
6
7 # When the block is exited, file.close() is called AUTOMATICALLY
```


Best Practice: The 'with' Statement

The 'with' Statement

Guarantees that the file is automatically closed when the indented block is exited, even if errors occur within the block.

```

1 # Recommended approach
2 with open("store.txt", "wt") as file:
3     # The file is open within this indented block
4     file.write("Hello\n")
5     file.write("World\n")
6
7 # When the block is exited, file.close() is called AUTOMATICALLY
    
```

Benefits

This is the standard, robust idiom for file handling in Python.

Reading from a File: Methods

Method 1: The Whole File ('.read()')

```
1 with open("names.txt", "rt") as file:
2     data = file.read() # Reads ENTIRE file into memory
```

- **Risk:** Can cause MemoryError if the file is larger than available RAM.

Method 2: Line by Line (Preferred)

```
1 with open("names.txt", "rt") as file:
2     for line in file: # Reads one line at a time (Generator)
3         print(line.strip()) # .strip() removes the newline '\n'
```

- **Advantage:** Memory efficient; suitable for files of any size.

Reading from a File: Methods

Method 1: The Whole File ('.read()')

```
1 with open("names.txt", "rt") as file:  
2     data = file.read() # Reads ENTIRE file into memory
```

- **Risk:** Can cause MemoryError if the file is larger than available RAM.

Method 2: Line by Line (Preferred)

```
1 with open("names.txt", "rt") as file:  
2     for line in file: # Reads one line at a time (Generator)  
3         print(line.strip()) # .strip() removes the newline '\n'
```

- **Advantage:** Memory efficient; suitable for files of any size.

Code Example: 'store.py' (Text Handling) I

Reviewing the provided example demonstrates these concepts.

Listing 1: store.py (test_write function)

```
1 def test_write():
2     # Traditional open/close (used here for demonstration, but '
3     #     with' is better)
4     file = open("store.txt", "wt")
5     file.write("Hello\n")
6     file.write("World\n")
7     file.close() # Essential for flushing the buffer
8
9     # Reading back using 'with' (Best practice)
10    with open("store.txt", "rt") as file2:
```

Code Example: 'store.py' (Text Handling) II

```
10     # Efficient line-by-line iteration
11     for line in file2:
12         # Note: print() adds its own newline, so output might
            appear
13         # double-spaced unless the line's newline is stripped.
14         print(line)
```

Storing Binary Data

Storing Binary Data

The Inefficiency of Text for Numerical Data

Storing numerical data in text files is often inefficient in terms of both storage space and processing time.

The Overhead of Conversion

To store a number as text, it must be converted to a string ('str(i)') when writing and parsed back ('int(s)') when reading. This consumes CPU cycles.

The Overhead of Storage Space

The string representation often takes more space than the raw numerical data.

Example: The integer 1,000,000

- As a standard 32-bit integer: Requires 4 bytes.
- As text ("1000000" + newline \n): Requires 8 bytes.

The Inefficiency of Text for Numerical Data

Storing numerical data in text files is often inefficient in terms of both storage space and processing time.

The Overhead of Conversion

To store a number as text, it must be converted to a string ('str(i)') when writing and parsed back ('int(s)') when reading. This consumes CPU cycles.

The Overhead of Storage Space

The string representation often takes more space than the raw numerical data.

Example: The integer 1,000,000

- As a standard 32-bit integer: Requires 4 bytes.
- As text ("1000000" + newline \n): Requires 8 bytes.

The Solution: Binary Mode

For efficiency, we should store non-text data in its native binary representation.

Binary Mode ('b')

We use the 'b' mode specifier instead of 't'.

- 'wb': Write Binary
- 'rb': Read Binary

The Difference

In binary mode, Python reads and writes raw 'bytes' objects directly, without applying any text encoding or decoding.

The Solution: Binary Mode

For efficiency, we should store non-text data in its native binary representation.

Binary Mode ('b')

We use the 'b' mode specifier instead of 't'.

- 'wb': Write Binary
- 'rb': Read Binary

The Difference

In binary mode, Python reads and writes raw 'bytes' objects directly, without applying any text encoding or decoding.

The Solution: Binary Mode

Binary Mode ('b')

We use the 'b' mode specifier instead of 't'.

- 'wb': Write Binary
- 'rb': Read Binary

The Difference

In binary mode, Python reads and writes raw 'bytes' objects directly, without applying any text encoding or decoding.

The Challenge

If we want to write an integer to a binary file, we must explicitly tell Python how to represent that integer as a sequence of bytes. We must define the format.

Defining the Binary Format

Computers can represent integers in multiple ways. We must specify the format.

1. Size (Number of Bytes)

How many bytes should be used for each integer?

- E.g., 4 bytes (32-bit integer).

2. Byte Order (Endianness)

In what order should the bytes be arranged in the file?

- **Little-Endian:** The least significant byte is stored first (Common in x86/Intel/AMD architectures).
- **Big-Endian:** The most significant byte is stored first.

Consistency is Key

Defining the Binary Format

Computers can represent integers in multiple ways. We must specify the format.

1. Size (Number of Bytes)

How many bytes should be used for each integer?

- E.g., 4 bytes (32-bit integer).

2. Byte Order (Endianness)

In what order should the bytes be arranged in the file?

- **Little-Endian:** The least significant byte is stored first (Common in x86/Intel/AMD architectures).
- **Big-Endian:** The most significant byte is stored first.

Consistency is Key

Defining the Binary Format

1. Size (Number of Bytes)

How many bytes should be used for each integer?

- E.g., 4 bytes (32-bit integer).

2. Byte Order (Endianness)

In what order should the bytes be arranged in the file?

- **Little-Endian:** The least significant byte is stored first (Common in x86/Intel/AMD architectures).
- **Big-Endian:** The most significant byte is stored first.

Consistency is Key

The format chosen for writing must match the format used for reading.

Writing Binary Data in Python

Python provides the `int.to_bytes()` method for this explicit conversion.

The `int.to_bytes()` Method

Syntax: `integer.to_bytes(length, byteorder)`

```

1 i = 1000000
2
3 # Convert 'i' into 4 bytes (32-bit) using little-endian order
4 binary_data = i.to_bytes(4, byteorder="little")
5
6 # Write the resulting bytes object to the file
7 with open("nums.bin", "wb") as file:
8     file.write(binary_data)

```

Writing Binary Data in Python

The `int.to_bytes()` Method

Syntax: `integer.to_bytes(length, byteorder)`

```
1 i = 1000000
2
3 # Convert 'i' into 4 bytes (32-bit) using little-endian order
4 binary_data = i.to_bytes(4, byteorder="little")
5
6 # Write the resulting bytes object to the file
7 with open("nums.bin", "wb") as file:
8     file.write(binary_data)
```

Efficiency

Storing 10 such numbers now takes exactly 40 bytes ($10 * 4$), compared to the 80 bytes

Reading Binary Data in Python

Reading binary data involves reversing the process.

Reading Raw Bytes

In binary mode, 'file.read(n)' reads exactly 'n' bytes and returns a 'bytes' object.

```
1 with open("nums.bin", "rb") as file:
2     # Read exactly 4 bytes
3     b = file.read(4)
```

Conversion: int.from_bytes()

We must convert the bytes object back into an integer, specifying the same byte order used for writing.

```
6     # Convert bytes 'b' using little-endian order
7     i = int.from_bytes(b, byteorder="little")
```

Reading Binary Data in Python

Reading Raw Bytes

In binary mode, 'file.read(n)' reads exactly 'n' bytes and returns a 'bytes' object.

```
1 with open("nums.bin", "rb") as file:
2     # Read exactly 4 bytes
3     b = file.read(4)
```

Conversion: int.from_bytes()

We must convert the bytes object back into an integer, specifying the same byte order used for writing.

```
6     # Convert bytes 'b' using little-endian order
7     i = int.from_bytes(b, byteorder="little")
8     print(i) # 1000000
```

Random Access: Seeking in Files

A crucial feature of file I/O, especially crucial for binary files with fixed-size records, is the ability to jump to specific locations.

The File Cursor

When a file is opened, the read/write position (cursor) starts at byte 0. Reading or writing advances the cursor.

Key Methods

`file.seek(offset)`: Jumps to the specified byte offset from the beginning of the file.

`file.tell()`: Returns the current byte position of the cursor.

Random Access: Seeking in Files

A crucial feature of file I/O, especially crucial for binary files with fixed-size records, is the ability to jump to specific locations.

The File Cursor

When a file is opened, the read/write position (cursor) starts at byte 0. Reading or writing advances the cursor.

Key Methods

`file.seek(offset)`: Jumps to the specified byte offset from the beginning of the file.

`file.tell()`: Returns the current byte position of the cursor.

Random Access: Seeking in Files

The File Cursor

When a file is opened, the read/write position (cursor) starts at byte 0. Reading or writing advances the cursor.

Key Methods

`file.seek(offset)`: Jumps to the specified byte offset from the beginning of the file.

`file.tell()`: Returns the current byte position of the cursor.

Application

This ability to perform random access (seeking) is fundamental for building complex systems like databases, which need to read and update specific records without reading the entire file.

Code Example: 'store.py' (Binary Handling) I

Listing 2: store.py (test_nums function - Writing)

```
1 def test_nums():
2     # Use 'with' for writing as well (best practice)
3     # The original example used manual open/close, shown here for
      fidelity:
4     file = open("nums.bin", "wb") # Write Binary mode
5     for i in range(1000000, 1000010):
6         # Convert to 4 bytes, little-endian
7         binary_data = i.to_bytes(4, byteorder="little")
8         file.write(binary_data)
9     file.close()
```

Code Example: 'store.py' (Binary Handling) II

Listing 3: store.py (test_nums function - Reading/Seeking)

```

1  with open("nums.bin", "rb") as file2: # Read Binary mode
2      # Goal: Read the 3rd integer (1000002)
3
4      # Skip first 8 bytes (2 integers * 4 bytes/integer)
5      file2.seek(8)
6
7      b = file2.read(4) # Read the next 4 bytes
8      # Convert back using the same format
9      i = int.from_bytes(b, byteorder="little")
10     print(i)           # Prints 1000002
11     # Check position (8 bytes seeked + 4 bytes read)
12     print(file2.tell()) # Prints 12

```

Code Example: 'store.py' (Binary Handling) III

Storage at Scale (Big Data)

Storage at Scale (Big Data)

Defining "Big Data"

The Technical Definition

In the context of algorithms and storage, "Big Data" applies when the dataset you need to process is significantly **larger than your computer's available RAM**.

The Implication

It is impossible to load the entire dataset into memory (e.g., using `file.read()`).

The Required Approach: Chunking (Online Algorithms)

The data must be processed sequentially or in chunks: Read a chunk, process it, write results, repeat.

Defining "Big Data"

The Technical Definition

In the context of algorithms and storage, "Big Data" applies when the dataset you need to process is significantly **larger than your computer's available RAM**.

The Implication

It is impossible to load the entire dataset into memory (e.g., using `file.read()`).

The Required Approach: Chunking (Online Algorithms)

The data must be processed sequentially or in chunks: Read a chunk, process it, write results, repeat.

Defining "Big Data"

The Technical Definition

In the context of algorithms and storage, "Big Data" applies when the dataset you need to process is significantly **larger than your computer's available RAM**.

The Implication

It is impossible to load the entire dataset into memory (e.g., using `'file.read()'`).

The Required Approach: Chunking (Online Algorithms)

The data must be processed sequentially or in chunks: Read a chunk, process it, write results, repeat.

Historical Context: External Sorting

The challenge of processing data larger than memory is not new.

Example: 1960s Computing

Early computers had very little RAM but processed large datasets stored on magnetic tapes.

The Merge Sort Algorithm

The Merge Sort algorithm (discussed in previous weeks) is well-suited for this scenario (External Sorting).

- The Merge step only requires the current item from two input lists (tapes) to be in memory.
- It reads inputs sequentially and writes the merged output sequentially.
- Allows sorting massive amounts of data with minimal RAM.

Historical Context: External Sorting

The challenge of processing data larger than memory is not new.

Example: 1960s Computing

Early computers had very little RAM but processed large datasets stored on magnetic tapes.

The Merge Sort Algorithm

The Merge Sort algorithm (discussed in previous weeks) is well-suited for this scenario (External Sorting).

- The Merge step only requires the current item from two input lists (tapes) to be in memory.
- It reads inputs sequentially and writes the merged output sequentially.
- Allows sorting massive amounts of data with minimal RAM.

Modern Example: Search Engines

Search engines like Google exemplify the challenges and solutions for handling data at an extreme scale.

The Task: Indexing the Web

- 1 Download the web (e.g., 30 trillion+ pages), refreshing frequently.
- 2 Build an index (a structure allowing fast lookups of words).
- 3 Serve user queries rapidly (e.g., $\leq$ 0.5 seconds).

The Scale of the Index

Google's index is estimated to be around 100 million gigabytes (100 Petabytes).

Modern Example: Search Engines

Search engines like Google exemplify the challenges and solutions for handling data at an extreme scale.

The Task: Indexing the Web

- 1 Download the web (e.g., 30 trillion+ pages), refreshing frequently.
- 2 Build an index (a structure allowing fast lookups of words).
- 3 Serve user queries rapidly (e.g., $\leq$ 0.5 seconds).

The Scale of the Index

Google's index is estimated to be around 100 million gigabytes (100 Petabytes).

The Intractability of a Single Server

Hardware Limitations vs. Data Scale

- Index Size: $\approx 100,000,000$ GB (100 PB).
- High-end Server RAM: $\approx 10,000$ GB (10 TB).

The Problem

The index is thousands of times larger than the RAM of even the most powerful single server. It cannot fit on one machine.

The Requirement

We would need approximately 10,000 high-end servers just to hold a single copy of the index.

The Intractability of a Single Server

Hardware Limitations vs. Data Scale

- Index Size: $\approx 100,000,000$ GB (100 PB).
- High-end Server RAM: $\approx 10,000$ GB (10 TB).

The Problem

The index is thousands of times larger than the RAM of even the most powerful single server. It cannot fit on one machine.

The Requirement

We would need approximately 10,000 high-end servers just to hold a single copy of the index.

The Intractability of a Single Server

Hardware Limitations vs. Data Scale

- Index Size: $\approx 100,000,000$ GB (100 PB).
- High-end Server RAM: $\approx 10,000$ GB (10 TB).

The Problem

The index is thousands of times larger than the RAM of even the most powerful single server. It cannot fit on one machine.

The Requirement

We would need approximately 10,000 high-end servers just to hold a single copy of the index.

The Solution: Sharding and Distribution

The only viable solution is to distribute the data and the workload across a massive cluster of servers (a data center).

Sharding

The index is divided into thousands of pieces, called **shards** (e.g., 10,000 shards).

- Each shard indexes a fraction of the web (e.g., 1/10,000th) for *all* possible words.
- One high-end server (or a small group) is responsible for one shard.

Replication

For reliability and performance (handling global traffic), each shard is replicated across many servers in different geographical locations.

The Solution: Sharding and Distribution

The only viable solution is to distribute the data and the workload across a massive cluster of servers (a data center).

Sharding

The index is divided into thousands of pieces, called **shards** (e.g., 10,000 shards).

- Each shard indexes a fraction of the web (e.g., 1/10,000th) for *all* possible words.
- One high-end server (or a small group) is responsible for one shard.

Replication

For reliability and performance (handling global traffic), each shard is replicated across many servers in different geographical locations.

Query Execution at Scale

The Distributed Query Process

When a user submits a search query:

- 1 The query is sent simultaneously to one server for every shard (e.g., 10,000 servers engage).
- 2 Each server searches its own local shard in parallel for the query terms.
- 3 Each server returns its top results to a front-end server.
- 4 The front-end server aggregates the results, ranks them (determines relevance), and sends the final page back to the user.

The Achievement

This massive parallelism allows searching 30 trillion pages, involving 10,000+ servers, in under half a second.

Query Execution at Scale

The Distributed Query Process

When a user submits a search query:

- 1 The query is sent simultaneously to one server for every shard (e.g., 10,000 servers engage).
- 2 Each server searches its own local shard in parallel for the query terms.
- 3 Each server returns its top results to a front-end server.
- 4 The front-end server aggregates the results, ranks them (determines relevance), and sends the final page back to the user.

The Achievement

This massive parallelism allows searching 30 trillion pages, involving 10,000+ servers, in under half a second.

Query Execution at Scale

The Distributed Query Process

When a user submits a search query:

- 1 The query is sent simultaneously to one server for every shard (e.g., 10,000 servers engage).
- 2 Each server searches its own local shard in parallel for the query terms.
- 3 Each server returns its top results to a front-end server.
- 4 The front-end server aggregates the results, ranks them (determines relevance), and sends the final page back to the user.

The Achievement

This massive parallelism allows searching 30 trillion pages, involving 10,000+ servers, in under half a second.

Query Execution at Scale

The Distributed Query Process

When a user submits a search query:

- 1 The query is sent simultaneously to one server for every shard (e.g., 10,000 servers engage).
- 2 Each server searches its own local shard in parallel for the query terms.
- 3 Each server returns its top results to a front-end server.
- 4 The front-end server aggregates the results, ranks them (determines relevance), and sends the final page back to the user.

The Achievement

This massive parallelism allows searching 30 trillion pages, involving 10,000+ servers, in under half a second.

Query Execution at Scale

The Distributed Query Process

When a user submits a search query:

- 1 The query is sent simultaneously to one server for every shard (e.g., 10,000 servers engage).
- 2 Each server searches its own local shard in parallel for the query terms.
- 3 Each server returns its top results to a front-end server.
- 4 The front-end server aggregates the results, ranks them (determines relevance), and sends the final page back to the user.

The Achievement

This massive parallelism allows searching 30 trillion pages, involving 10,000+ servers, in under half a second.

Data Privacy and Legal Obligations

Data Privacy and Legal Obligations

The Responsibility of Data Storage

While the technology of data storage is powerful, we must recognize that data has meaning and impacts people's lives.

Professional Ethics and Legal Duty

Protecting user privacy is a core responsibility of a professional computer scientist and is mandated by law.

The Legal Framework: GDPR

The General Data Protection Regulation (GDPR), along with national laws (like the UK Data Protection Act), governs the processing of personal information.

The Responsibility of Data Storage

While the technology of data storage is powerful, we must recognize that data has meaning and impacts people's lives.

Professional Ethics and Legal Duty

Protecting user privacy is a core responsibility of a professional computer scientist and is mandated by law.

The Legal Framework: GDPR

The General Data Protection Regulation (GDPR), along with national laws (like the UK Data Protection Act), governs the processing of personal information.

The Responsibility of Data Storage

Professional Ethics and Legal Duty

Protecting user privacy is a core responsibility of a professional computer scientist and is mandated by law.

The Legal Framework: GDPR

The General Data Protection Regulation (GDPR), along with national laws (like the UK Data Protection Act), governs the processing of personal information.

Definition: Personal Information

Any data related to an identifiable living person (names, addresses, IP addresses, behavioral data, etc.).

Key Principles of GDPR

If you process personal information, you must adhere to several core principles.

Lawful Grounds: You must have a legal basis to process the data (e.g., explicit user consent, a necessary contract). You cannot collect data arbitrarily.

Purpose Limitation: Data can only be used for the specific, defined purpose stated when it was collected.

Time Limitation (Data Retention): Data can only be kept for as long as necessary to fulfill that specific purpose.

Security: Data must be adequately secured against unauthorized access or loss.

Key Principles of GDPR

If you process personal information, you must adhere to several core principles.

Lawful Grounds: You must have a legal basis to process the data (e.g., explicit user consent, a necessary contract). You cannot collect data arbitrarily.

Purpose Limitation: Data can only be used for the specific, defined purpose stated when it was collected.

Time Limitation (Data Retention): Data can only be kept for as long as necessary to fulfill that specific purpose.

Security: Data must be adequately secured against unauthorized access or loss.

Key Principles of GDPR

If you process personal information, you must adhere to several core principles.

Lawful Grounds: You must have a legal basis to process the data (e.g., explicit user consent, a necessary contract). You cannot collect data arbitrarily.

Purpose Limitation: Data can only be used for the specific, defined purpose stated when it was collected.

Time Limitation (Data Retention): Data can only be kept for as long as necessary to fulfill that specific purpose.

Security: Data must be adequately secured against unauthorized access or loss.

Key Principles of GDPR

If you process personal information, you must adhere to several core principles.

Lawful Grounds: You must have a legal basis to process the data (e.g., explicit user consent, a necessary contract). You cannot collect data arbitrarily.

Purpose Limitation: Data can only be used for the specific, defined purpose stated when it was collected.

Time Limitation (Data Retention): Data can only be kept for as long as necessary to fulfill that specific purpose.

Security: Data must be adequately secured against unauthorized access or loss.

User Rights and Technical Implications

GDPR grants specific rights to users, which impose technical requirements on the systems we build.

Right to Remedy (Rectification)

Users have the right to correct incorrect data stored about them.

- **Technical Implication:** All personal data systems must be **mutable** (changeable). You cannot store personal data immutably.

Right of Access

Users can request a copy of all data you store about them.

- **Technical Implication:** You must be able to locate and extract all data associated with a specific user across all systems.

User Rights and Technical Implications

GDPR grants specific rights to users, which impose technical requirements on the systems we build.

Right to Remedy (Rectification)

Users have the right to correct incorrect data stored about them.

- **Technical Implication:** All personal data systems must be **mutable** (changeable). You cannot store personal data immutably.

Right of Access

Users can request a copy of all data you store about them.

- **Technical Implication:** You must be able to locate and extract all data associated with a specific user across all systems.

User Rights and Technical Implications (Cont.)

Right to be Forgotten (Erasure)

Users can request that their data be deleted.

- **Technical Implication:** You must be able to delete all data for a specific user. This includes data stored in indexes, logs, and **backups** (which is technically challenging).

Security in Practice

Data must be secured at all stages of its lifecycle:

- Secured in transit (e.g., using TLS).
- Secured in storage (at rest).
- Secured on backups.
- **Technical Implication:** This requires strong encryption and strict access controls (limiting who holds the encryption keys).

User Rights and Technical Implications (Cont.)

Right to be Forgotten (Erasure)

Users can request that their data be deleted.

- **Technical Implication:** You must be able to delete all data for a specific user. This includes data stored in indexes, logs, and **backups** (which is technically challenging).

Security in Practice

Data must be secured at all stages of its lifecycle:

- Secured in transit (e.g., using TLS).
- Secured in storage (at rest).
- Secured on backups.
- **Technical Implication:** This requires strong encryption and strict access controls (limiting who holds the encryption keys).

Summary I

Key Takeaways

Summary II

- RAM is volatile and OS-managed; persistent storage (HDDs/SSDs) is required for long-term data retention.
- The OS abstracts hardware complexity (blocks) into a File System (files as sequences of bytes).
- Python's 'open()' provides access. Always use the 'with' statement for safety (automatic closing and buffer flushing).
- Text mode ('t') handles encoding; Binary mode ('b') handles raw bytes.
- Binary storage is more efficient for numerical data but requires defining the format (size, endianness) using `to_bytes()/from_bytes()`.
- 'file.seek()' enables random access, crucial for databases.
- Handling "Big Data" (Data $\gg$ RAM) requires processing in chunks or massive distribution (sharding).